NUMERICAL ANALYSIS
GUIDED LAB 2

ROB TOMPKINS

Note, all the files associated with Guided Lab 2 are checked into the :
https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/GuidedLab2

Exercise 1. We've added the code files to github at the following links: gram_schmidt.m problem1.m
(a) We went about completing this problem by writing the following code:

```
function Q = gram_schmidt( X )
    % Q = gram_schmidt( X )
    % X is an d by n matrix
    % Q is an d by r matrix with 1 <= r <= n that is linearly independent
    % computes the Gram-Schmidt left decomposition matrix Q
    % By Rob Tompkins, 20260215
    [d,n] = size(X);
    m = min(d,n);
    R = zeros(m,n);
    Q = zeros(d,m);
    for i = 1:m
        v = X(:,i);
        for j = 1:i-1
            R(j,i) = Q(:,j)'*v;
            v = v-R(j,i)*Q(:,j);
        end
        R(i,i) = norm(v);
        Q(:,i) = v/R(i,i);
    end
    R(:,m+1:n) = Q'*X(:,m+1:n);
end
```

(b) We have two means by seeing that $X = [1 \quad 1 \quad 1; \quad 0 \quad 1 \quad 1; \quad 0 \quad 0 \quad 1]$ the Gram-Schmidt algorithm should yield I_3, the identity matrix on $\mathbb{R}^3$. The first way to notice is that it is an upper triangular matrix with no repeated columns. The other is that if we run the algorithm on the matrix X in matlab (see problem1.m for the matlab output) we get

$$Q1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Date: February 16, 2026.

(c) Similar to problem (b) it is easy to see that $X = [1 \quad 1 \quad 1; \quad 1 \quad 1 \quad 0; \quad 1 \quad 0 \quad 0]$ has determinant -1, and we can see that $\text{span}(X) = \text{span}\{e_1, e_2, e_3\}$ where $e_i \in \mathbb{R}^3$ is the vector that is all zeros except for 1 in the i^{th}-place. Furthermore, we ran our Gram-Schmidt process on X and calculated the norm over the column vectors and we got 1.

(d) We ran the Gram-Schmidt algorithm on the matrix $X = [2 \quad -1 \quad 0; \quad 0 \quad -1 \quad 2; \quad -1 \quad 0 \quad 0; -12-1]$ to get the following Q matrix

$$Q = \begin{bmatrix} 0.816496580927726 & 0.182574185835056 & 0.000000000000000 \\ 0 & -0.547722557505166 & 0.816496580927726 \\ -0.408248290463863 & -0.365148371670111 & -0.408248290463863 \\ -0.408248290463863 & 0.730296743340221 & 0.408248290463863 \end{bmatrix}.$$

If you look at the problem1.m file that we've linked to you'll see that each of the columns has length 1.

(e) It's quite impossible for Q to be orthogonal because $Q\top Q \in M_{3\times 3}(\mathbb{R})$ and $QQ^\top \in M_{4\times 4}(\mathbb{R})$, and $QQ^\top$ has non-zero entries off the diagonal.

Exercise 2. (a)For our "gs_factor" algorithm (given in the file gs_factor.m) that does the full decomposition $X = QR$ using the algorithm above we simply need to add R to the return values of the function. We can simply do this in the following manner:

```
function [Q, R] = gs_factor( X )
    % Q = gram_schmidt( X )
    % X is an m by n matrix
    % Q is an m by r matrix with 1 <= r <= n that is linearly independent
    % R is an n by n upper triangular matrix
    % computes the Gram-Schmidt left decomposition matrix X=QR
    % By Rob Tompkins, 20260215
    [d,n] = size(X);
    m = min(d,n);
    R = zeros(m,n);
    Q = zeros(d,m);
    for i = 1:m
        v = X(:,i);
        for j = 1:i-1
            R(j,i) = Q(:,j)'*v;
            v = v-R(j,i)*Q(:,j);
        end
        R(i,i) = norm(v);
        Q(:,i) = v/R(i,i);
    end
    R(:,m+1:n) = Q'*X(:,m+1:n);
end
```

(b) We used our matlab code for the following computation of the decomposition of the matrix in to $X = QR$:

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$Q = \begin{bmatrix} 0.577350269189626 & 0.408248290463863 & 0.707106781186547 \\ 0.577350269189626 & 0.408248290463863 & -0.707106781186548 \\ 0.577350269189626 & -0.816496580927726 & 0 \end{bmatrix}$$

$$R = \begin{bmatrix} 1.732050807568877 & 1.154700538379252 & 0.577350269189626 \\ 0 & 0.816496580927726 & 0.408248290463863 \\ 0 & 0 & 0.707106781186547 \end{bmatrix}$$

(c) We let $A = \text{rand}(100)$ in Matlab, and we ran $[Q1, R1] = \text{gs\_factor}(A)$. Then we ran the subsequent calculations for the sake of verifying various qualities of $Q1$ and $R1$:

```
[Q,R] = gs_factor([ 1 1 1; 1 1 0; 1 0 0 ])
Q*R

A = rand(100,100);
det(A)
[Q1,R1] = gs_factor(A);
det(Q1*R1)
norm(A)
norm(Q1*R1)

norm(Q1'*Q1 - eye(100))
norm(Q1*Q1' - eye(100))
norm(R - triu(R))
```

The results all came out as we would expect with $Q1$ being orthogonal, $R1$ being upper triangular, and $X = QR$. The calculations and answers can all be seen in the file problem2.m.

Exercise 3. We went about coding up the Householder QR decomposition. The files follow as: householderDecomposition.m, and problem3.m. The code follows as:

```
function [R,U] = householderDecomposition(A)
    % Householder reflections for QR decomposition.
    % [R,U] = house_qr(A) returns
    % R, the upper triangular factor, and
    % U, the reflector generators for use by house_apply.
    % By Cleve Moler, October 3, 2016
    % https://blogs.mathworks.com/cleve/2016/10/03/householder-reflections-and-the-qr-decomposition/
    H = @(u,x) x - u*(u'*x);
    [m,n] = size(A);
    U = zeros(m,n);
    R = A;
    for j = 1:min(m,n)
        u = house_gen(R(j:m,j));
        U(j:m,j) = u;
```

```
        R(j:m,j:n) = H(u,R(j:m,j:n));
        R(j+1:m,j) = 0;
    end
end

function u = house_gen(x)
    % u = house_gen(x)
    % Generate Householder reflection.
    % u = house_gen(x) returns u with norm(u) = sqrt(2), and
    % H(u,x) = x - u*(u'*x) = -+ norm(x)*e_1.

    % Modify the sign function so that sign(0) = 1.
    sig = @(u) sign(u) + (u==0);

    nu = norm(x);
    if nu ~= 0
        u = x/nu;
        u(1) = u(1) + sig(u(1));
        u = u/sqrt(abs(u(1)));
    else
        u = x;
        u(1) = sqrt(2);
    end
end
```

and some quick runs of the code against our matrix:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

give the decomposition QR in the following form, which we found suspect because we had the following:

$$Q = \begin{bmatrix} -1.732050807568877 & -1.154700538379251 & -0.577350269189626 \\ 0 & -0.816496580927726 & -0.408248290463863 \\ 0 & 0 & 0.707106781186547 \end{bmatrix}$$

$$R = \begin{bmatrix} 1.255926060399109 & 0 & 0 \\ 0.459700843380983 & 1.121971053593862 & 0 \\ 0.459700843380983 & -0.860918669153759 & -1.414213562373095 \end{bmatrix}$$

$$QR = R = \begin{bmatrix} -2.971552964179200 & -0.798488954244471 & 0.816496580927726 \\ -0.563016250305247 & -0.564616954248820 & 0.577350269189626 \\ 0.325057583671868 & -0.608761429008720 & -1.000000000000000 \end{bmatrix}$$

We also computed the values for (c) from exercise 2, but found inconsistencies leaving us suspect with the algorithm or its implementations. That said, the blog had some salient points about how these decompositions are necessarily approximate because of round off error and depending on how close to colinear our columns are we

Exercise 4. As specified in the guided lab we are working with the fact that for solving a system of lienar equations we can use the following breakdown because of our QR-decomposition:

$$\begin{aligned} Ax &= b \\ QRx &= b \\ Q^\top QRx &= Q^\top b \\ Rx &= Q^\top b \end{aligned}$$

So for us to effectively solve our equations we have two mechanisms, the first being the decomposition, the second being solving the upper triangular matrix problem. So our solution which we have in files problem4.m, and qrSolve.m.

The interesting bit is the implementation of "qrSolve.m" which follows as:

```
function x = qrSolve(Q, R, b)
    % x = qrSolve(Q, R, b)
    % Q - an n by n matrix that is the left hand side of a QR decomposition
    % R - an n by n matrix that is the right hand side of the QR
    % b - the vector on the right hand side of our equation Xx=b or QRx = b
    % The goal here is to solve our linear set of equations relying upon a
    % QR decomposition. This is equivalent to writing an algorithm
    % for solving an upper triangular equation because our equation is also
    % seen as Q'QRx = Q'b => Rx = Q'b
    % by Rob Tompkins, 20260215
    n = length(b);
    V = Q'*b;
    x = zeros(n,1);
    x(n) = V(n)/R(n,n);
    for k=n-1:-1:1
        x(k) = (V(k) - R(k, k+1:n) * x(k+1:n)) / R(k, k);
    end
end
```

A quick run of the following code (which was given by the problem)

```
n=7;
A=magic(n);
x=[1:n]';
b=A*x;
[Q,R]=gs_factor(A);
x1 = qrSolve(Q,R,b);
norm(x - x1)
```

yields the result that $\|x - x_1\| = 4.111978925869794 \times 10^{-14}$ effectively zero so our answer does work.

Exercise 5. Generally we are concerned with the decomposition of $A \in \mathbf{M}_{m\times n}(\mathbb{C})$ into three matrices U, S, and V such that

$$A = USV^*$$

where $U \in \mathbf{M}_{m\times m}(\mathbb{C})$ with $U^*U = UU^* = I_m$ and $V \in \mathbf{M}_{n\times n}(\mathbb{C})$ with $V^*V = VV^* = I_n$ and S be an $m \times n$ matrix with

$$S = [s_{kl}] \begin{cases} \sigma_k & \text{for } k = l \\ 0 & \text{for } k \neq l \end{cases}$$

with $\sigma_k \geq \sigma_{k-1}$ for $k = 2, \ldots, r$. The number r is the rank of A and $r \leq \min\{m, n\}$. Note that the singular values σ_k are positive real numbers and they are the eigenvalues of the hermitian matrix A^*A. Now this curiously provides us a mechanism for "solving" the equation $Ax = b$ despite the fact that A is not a square matrix. Before we begin we want to define the following matrix S^+

$$S^+ = [s_{kl}]^+ = \begin{cases} \frac{1}{\sigma_k} & \text{for } k = l \\ 0 & \text{for } k \neq l. \end{cases}$$

We can then proceed with the following algebraic manipulation to solve for x in $Ax = b$

$$\begin{aligned} Ax &= b \\ USV^*x &= b \\ U^*USV^*x &= U^*b \\ SV^*x &= U^*b \\ S^+SV^*x &= S^+U^*b \\ V^*x &= S^+U^*b \\ VV^*x &= VS^+U^*b \\ x &= VS^+U^*b. \end{aligned}$$

So we now have a kind of solution for x.

Exercise 6. Our goal was to take the photo in Figure 1 and decompose it using SVD.

We have been given the linked file M515GL2ex6gh.m for the purposes of computing all the different pieces of the exercise we used the following code:

```
%%
rgbGH=imread('GraceHopper.jpg');
```

FIGURE 1. Photo of Grace Hopper to be used as a matrix

```
figure();
image(rgbGH), axis image;  %plot color image

bwGH=rgb2gray(rgbGH); %convert to grayscale
imGH=double(bwGH); %convert from unsigned integers to double for calculutions


%plt grayscale image
figure();
colormap(gray(256));
image(imGH);
daspect([1 1 1]) %this preserves aspect ratio, could also use "axis image"
title('Original');


%Compute singular value decomposition
[U S V]=svd(imGH);

%plot singular values on semilog scale.
%Notice how quickly the magnitude drops.
figure;
semilogy(diag(S))
ylabel('Singular Values')
xlabel('n')


%pick number of singular values to use for reconstructing image
Nsvals=[200, 100, 50, 25, 10];

%plot "compressed" images by only include the largest ns singular values
for jj=1:length(Nsvals)
    ns=Nsvals(jj);
```

```
    imNs=U(:,1:ns)*S(1:ns,1:ns)*V(:,1:ns)';

    figure();
    colormap(gray(256));
    image(imNs);
    daspect([1 1 1])
    title([num2str(ns)  ' singular values']);

end
```

Running this code on the image, stored in our repo, "GraceHopper.jpg", when we had only 25 singular values we resulted in the following image compression (see Figure 2)

FIGURE 2. SVD Compression of the GraceHopper.jpg image with 25 singular values

We also got a semi-log plot of our singular values with respect to n, given in Figure 3

Exercise 7. We conducted a similar study as in exercise 6, but instead we used the photo in Figure 4. This yielded the following figures 5 and 6.

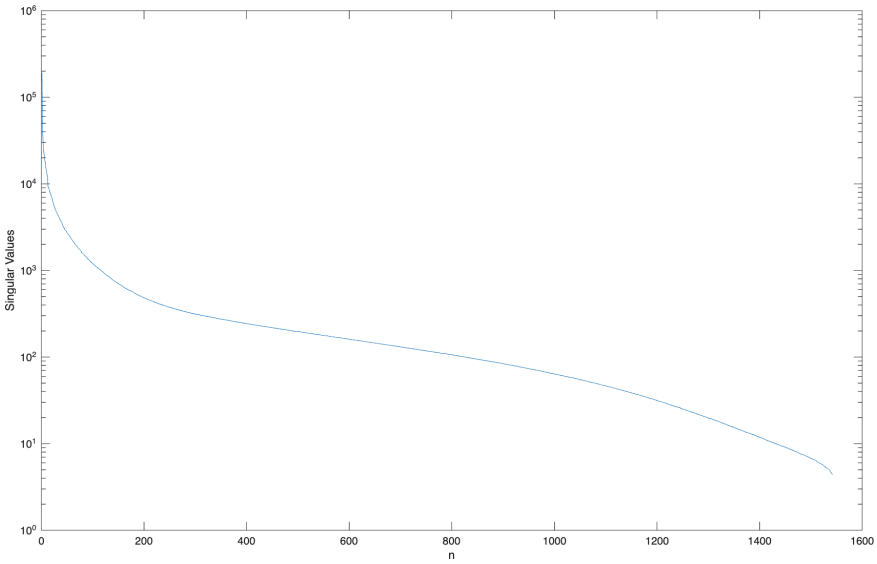

FIGURE 3. Semi-log plot of our singular values

FIGURE 4. An arbitrary photo for SVD decomposition

FIGURE 5. SVD Compression of the photo from Figure 4 image with 50 singular values

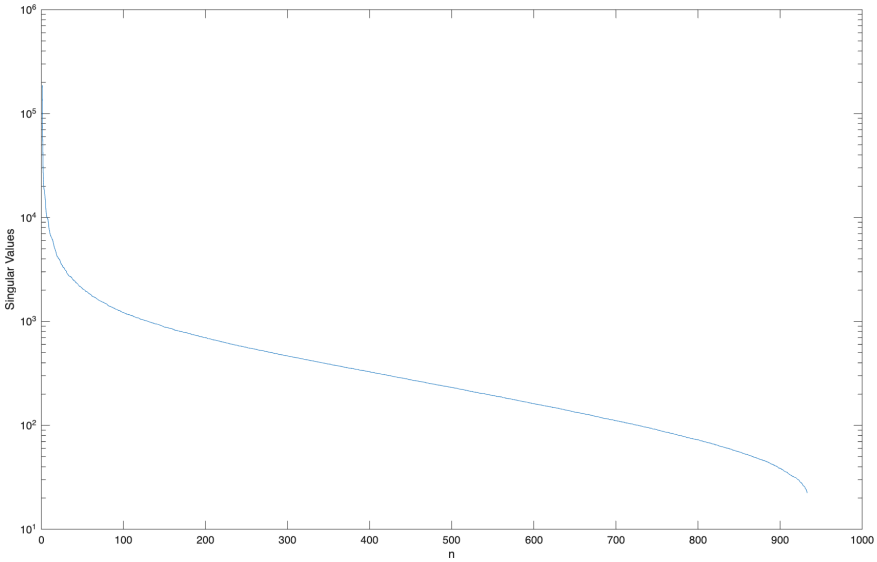

FIGURE 6. Semi-log plot of our singular values from the waterfall photo